11: ConfigMaps and Secrets

Most business applications comprise two main parts.

- The application
- The configuration

A simple example is a web server such as NGINX or httpd (Apache). Neither are very useful without a configuration. However, when you combine them with a configuration, they become extremely useful.

In the past, we coupled the application and the configuration into a single easy-to-deploy unit. As we moved into the early days of cloud-native microservices applications, we brought this model with us. However, it's an *anti-pattern* in the cloud-native world. You should de-couple the application and the configuration, bringing benefits such as:

- Re-usable application images
- Simpler development and testing
- Simpler and fewer disruptive changes

We'll explain all of these, and more, as we go through the chapter.

The chapter is divided as follows:

- The big picture
- ConfigMap theory
- Hands-on with ConfigMaps
- Hands-on with Secrets

The big picture

As already mentioned, most applications comprise an application binary and a configuration. This doesn't change with apps on Kubernetes. You build and store them separately, and bring them together at run-time.

Let's consider an example to understand some of the benefits…

Quick example

Imagine you work for a company that deploys modern applications to Kubernetes, and you have three distinct environments:

- Dev
- Test

- Prod

Your developers write and update applications. Initial testing is performed in the *dev* environment, further testing is done in the *test* environment where more stringent rules and the likes are applied. Finally, stable components graduate to the *prod* environment.

However, each environment has subtle differences. These include things such as number of nodes, configuration of nodes, network and security policies, different sets of credentials and certificates, and more.

You currently package each application microservice with its configuration baked into the container (the application and configuration are packaged as a single artefact). With this in mind, you have to perform all of the following for every business application:

- *build* three distinct images (one for dev, one for test, one for prod)
- *store* the images in three distinct repositories (dev, test, prod)
- *run* each version of the image in a specific environment (dev in dev, test in test, prod in prod)

Every time you change the config of an app, even small changes like fixing a typo, you need to build and package an entirely new image and perform some type of update to the entire app.

Analysing the example

There are several drawbacks to the approach of storing the application and its configuration as a single artefact (container image).

As your *dev*, *test*, and *prod* environments have different characteristics, each environment needs its own image. A *dev* or *test* image won't work in the *prod* environment because of things like different security credentials. This requires extra work to create and maintain 3x copies each application. This complicates matters and increases the chances of misconfiguration, including things that work in dev and test but not in prod.

You also have to store 3x images in 3 distinct repositories. Plus, you need to be very careful about permissions to repositories. This is because your *prod* images contain sensitive configuration data, sensitive passwords, and sensitive encryption keys. You probably don't want dev and test engineers having access to prod images.

It's also harder to troubleshoot an issue if you push an update that includes both an application binary update as well as a configuration update. If the two are tightly coupled, it's harder to isolate the fault. Also, if you need to make a minor configuration change (for example fix a typo on a web page) you need to re-package, re-test, and re-deploy the entire application **and** configuration.

None of this is ideal.

What it looks like in a de-coupled world

Now consider you work for the same company, but this time your application and its configuration are de-coupled. This time.

- You *build* a single application image that's shared across all three environments
- You *store* a single image in a single repository
- You *run* a single version of each image in all environments

To make this work, you build your application images as generically as possible with no embedded configuration. You then create and store configurations in separate objects, and apply a configuration to the application at when you run it. For example, you have a single copy of a web server that you can deploy to all three environments. When you deploy it to *prod* you apply the *prod* configuration. When you run it in *dev*, you apply the *dev* configuration…

In this model, you create and test a single version of each application image that you store in a single repository. All staff can have access to the image repository as there's no sensitive data in the images. Finally, you can easily push changes to the application and its configuration independent of each other – updating a simple typo in a config no longer requires the entire application binary and image to be rebuilt and re-deployed.

You can even re-use images across different apps. For example, a hardened stripped-down NGINX image can be used by lots of different apps – just load different configs at run-time.

Let's see how Kubernetes makes this possible…

ConfigMap theory

Kubernetes provides an object called a ConfigMap (CM) that lets you store configuration data outside of a Pod. It also lets you dynamically inject the config into a Pod at run-time.

> **Note:** When we use the term *Pod* we mean container. After all, it is ultimately a container that receives the configuration data and runs the app.

ConfigMaps are first-class objects in the Kubernetes API under the *core* API group, and they're `v1`. This tells us a lot of things:

1. They're stable (`v1`)
2. They've been around for a while (the fact that they're in the core API group)
3. You can operate on them with the usual `kubectl` commands
4. They can be defined and deployed via the usual YAML manifests

ConfigMaps are typically used to store non-sensitive configuration data such as:

- Environment variables
- Entire configuration files (things like web server configs and database configs)
- Hostnames
- Service ports
- Accounts names

You should **not** use ConfigMaps to store sensitive data such as certificates and passwords. Kubernetes provides a different object, called a *Secret*, for storing sensitive data. Secrets and ConfigMaps are very similar in design and implementation, the major difference is that Kubernetes takes steps to obscure the data stored in Secrets. It makes no such efforts to obscure data stored in ConfigMaps.

You'll see Secrets at the end of the chapter.

How ConfigMaps work

At a high-level, a ConfigMap is a place to store configuration data that can be seamlessly injected into containers at runtime. As far as any app is concerned, there is no ConfigMap. The config data simply surfaces where it's expected and the app has no idea it was put there by a ConfigMap.

Let's look a bit closer…

Behind the scenes, ConfigMaps are a map of key/value pairs and we call each key/value pair an *entry*.

- **Keys** are an arbitrary name that can be created from alphanumerics, dashes, dots, and underscores
- **Values** can contain anything, including multiple lines with carriage returns
- Keys and values are separated by a colon – `key:value`

Some simple examples might be:

- db-port:13306
- hostname:msb-prd-db1

More complex examples can store entire configuration files like this one:

```
key: conf
value:

directive in;
main block;
http {
  server {
    listen        80 default_server;
    server_name   *.nigelpoulton.com;
    root          /var/www/nigelpoulton.com;
    index         index.html

    location / {
      root   /usr/share/nginx/html;
      index  index.html;
    }
  }
}
```

Once data is stored in a ConfigMap, it can be injected into containers at run-time via any of the following methods:

- Environment variables
- Arguments to the container's startup command
- Files in a volume

All of the methods work seamlessly with existing applications. In fact, all an application sees is its configuration data in either; an environment variable, an argument to a startup command, or a file in a filesystem. The application is unaware the data originally came from a ConfigMap.

Figure 11.1 shows how the pieces connect.

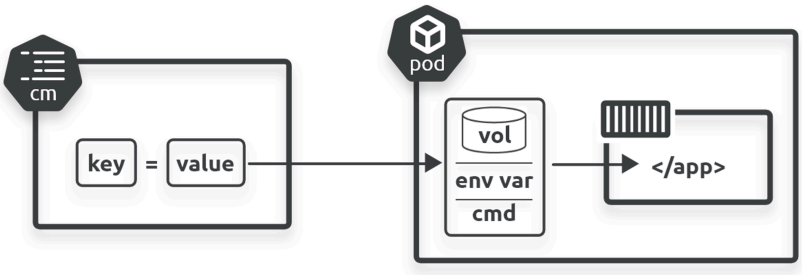

Figure 11.1

The most flexible of the three methods is the *volume* option, whereas the most limited is the *startup command*. You'll look at each in turn, but before we do that let's quickly mention *Kubernetes-native* applications.

ConfigMaps and Kubernetes-native apps

A *Kubernetes-native application* is one that knows it's running on Kubernetes and can talk to the Kubernetes API. As a result, they can access ConfigMap data directly via the API without needing things like environment variables and volumes. This can simplify application configuration, but the application will only run on Kubernetes (Kubernetes lock-in). At the time of writing, Kubernetes-native applications are rare.

Hands-on with ConfigMaps

You'll need a Kubernetes cluster and the lab files from the book's GitHub repo if you want to follow along.

```
$ git clone https://github.com/nigelpoulton/TheK8sBook.git
Cloning into 'TheK8sBook'...
```

If you don't know how to use `git`, or can't install it, you can just go to the repo and copy the text from the relevant files into files with the same name on your local machine.

Be sure to run all of the following commands from within the `configmaps` folder.

As with most Kubernetes objects, you can create them imperatively and declaratively. We'll look at the imperative method first.

Creating ConfigMaps imperatively

The command to imperatively create a ConfigMap is `kubectl create configmap`, but you can shorten `configmap` to `cm`. The command accepts two sources of data:

- Literal values on the command line (`--from-literal`)
- Files (`--from-file`)

Run the following command to create a ConfigMap called `testmap1` with two map entries from literal command-line values.

```
$ kubectl create configmap testmap1 \
  --from-literal shortname=AOS \
  --from-literal longname="Agents of Shield"
```

The following `kubectl describe` command shows how the two entries are stored in the map.

```
$ kubectl describe cm testmap1
Name:         testmap1
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
longname:
----
Agents of Shield
shortname:
----
AOS
Events:  <none>
```

You can see the object is essentially a map of key/value pairs dressed up as a Kubernetes object.

The next command creates a ConfigMap from a file called `cmfile.txt`. It assumes you have a local file called `cmfile.txt` in your working directory. The file contains the following single line of text and is available in the `configmaps` directory of the book's GitHub repo.

```
Kubernetes FTW
```

Run this command to create the ConfigMap from the contents of the file. Notice that the command uses the `--from-file` argument instead of `--from-literal`.

```
$ kubectl create cm testmap2 --from-file cmfile.txt
configmap/testmap2 created
```

Inspecting ConfigMaps

ConfigMaps are first class API objects. This means you can inspect and query them in the same way as any other API object.

List all ConfigMaps in the current Namespace.

```
$ kubectl get cm
AME       DATA   AGE
testmap1   2      11m
testmap2   1      2m23s
```

The following `kubectl describe` command shows the following interesting info about the `testmap2` map:

- A single map entry was created
- The name of the entry's key is the name of the file (`cmfile.txt`)
- The entry's value is the contents of the file

```
$ kubectl describe cm testmap2
Name:         testmap2
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
cmfile.txt:           <<-- key
----
Kubernetes FTW        <<-- value
Events:  <none>
```

You can also see the entire object by using the `-o yaml` flag with `kubectl get`.

```
$ kubectl get cm testmap1 -o yaml
apiVersion: v1
data:
  longname: Agents of Shield
  shortname: AOS
kind: ConfigMap
metadata:
  creationTimestamp: "2021-02-09T10:09:46Z"
  managedFields:
  <Snip>
    manager: kubectl-create
    operation: Update
    time: "2021-02-09T10:09:46Z"
  name: testmap1
  namespace: default
  resourceVersion: "311949"
  uid: 56321c4f-52f5-4ff3-90cd-22e018588065
```

An interesting thing to note is that ConfigMap objects don't have the concept of state (desired state and actual state). This is why they have a `data` block instead of `spec` and `status` blocks.

Let's find out how to create a ConfigMap declaratively before you use one to inject data into a Pod.

Creating ConfigMaps declaratively

The following ConfigMap manifest defines two map entries; `firstname` and `lastname`. It's available in the book's GitHub repo under the `configmaps` folder called `multimap.yml`. Alternatively, you can create an empty file and practice writing your own manifests from scratch.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: multimap
data:
  given: Nigel
  family: Poulton
```

You can see that a ConfigMap manifest has the normal `kind` and `apiVersion` fields, as well as the usual `metadata` section. However, as previously mentioned, they don't have a `spec` section. Instead, they have a `data` section that defines the map of key/values.

You can deploy it with the following command (the command assumes you have a copy of the file in your working directory called `multimap.yml`).

```
$ kubectl apply -f multimap.yml
configmap/multimap created
```

This next YAML looks slightly more complicated but it's actually not – it creates a ConfigMap with just a single map entry in the `data` block. It looks more complicated because the *value* portion of the map entry is a full configuration file.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: test-conf
data:
  test.conf: |
    env = plex-test
    endpoint = 0.0.0.0:31001
    char = utf8
    vault = PLEX/test
    log-size = 512M
```

The previous YAML file inserts a pipe character (|) after the name of the entry's *key* property. This tells Kubernetes that everything following the pipe is to be treated as a single literal value. Therefore, the ConfigMap object is called `test-conf` and it contains a single map entry as follows:

Key	Value
test.conf	env = plex-test endpoint = 0.0.0.0:31001 char = utf8 vault = PLEX/test log-size = 512M

You can deploy the previous CM with the following `kubectl` command. It assumes you have a local copy of the file called `singlemap.yml`.

```
$ kubectl apply -f singlemap.yml
configmap/test-conf created
```

List and describe the `multimap` and `test-conf` ConfigMaps you just created. The following shows the output of a `kubectl describe` against the `test-conf` map.

```
$ kubectl describe cm test-conf
Name:         test-conf
Namespace:    default
Labels:       <none>
Annotations:  <none>
Data
====
test.conf:
----
env = plex-test
endpoint = 0.0.0.0:31001
char = utf8
vault = PLEX/test
log-size = 512M
Events:  <none>
```

ConfigMaps are extremely flexible and can be used to insert complex configurations, including JSON files and even scripts, into containers at run-time.

Injecting ConfigMap data into Pods and containers

You've seen how to imperatively and declaratively create ConfigMap objects and populate them with data. Now let's see how to get that data into applications running in containers.

There are three main ways to inject ConfigMap data into a container:

- As environment variables
- As arguments to container startup commands
- As files in a volume

Let's look at each.

ConfigMaps and environment variables

A common way to get ConfigMap data into a container is via environment variables. You create the ConfigMap, then you map its entries into environment variables in the container section of a Pod template. When the container is started, the environment variables appear in the container as standard Linux or Windows environment variables.

Figure 11.2. shows this.

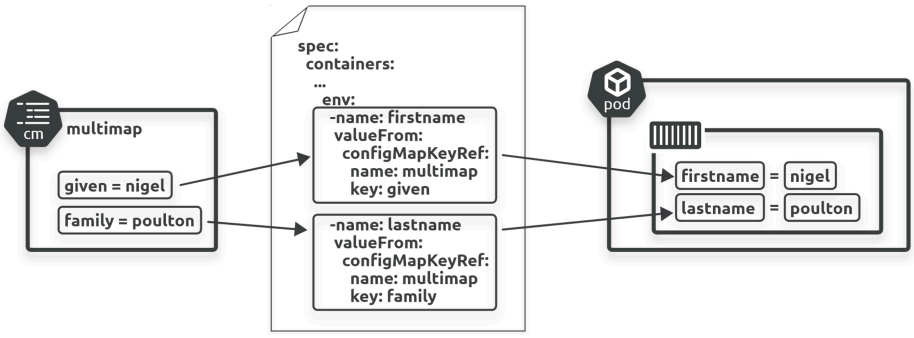

Figure 11.2

You already have a ConfigMap called `multimap` that has two entries:

- given=Nigel
- family=Poulton

The following Pod manifest deploys a single container that creates two environment variables in the container.

- FIRSTNAME: Maps to the `given` entry
- LASTNAME: Maps to the `family` entry

```
apiVersion: v1
kind: Pod
<Snip>
spec:
  containers:
    - name: ctr1
      env:
        - name: FIRSTNAME
          valueFrom:
            configMapKeyRef:
              name: multimap
              key: given
        - name: LASTNAME
          valueFrom:
            configMapKeyRef:
```

```
            name: multimap
            key: family
<Snip>
```

When the Pod is scheduled and the container started, `FIRSTNAME` and `LASTNAME` will be created as standard Linux environment variables inside the container. Applications can use these like regular environment variables – because they are!

Run the following commands to deploy a Pod from the `envpod.yml` file and list environment variables that include the `NAME` string in their name. This will list the `FIRSTNAME` and `LASTNAME` variables and you'll see they're populated with the values from the `multimap` ConfigMap.

```
$ kubectl apply -f envpod.yml
pod/envpod created

$ kubectl exec envpod -- env | grep NAME
HOSTNAME=envpod
FIRSTNAME=Nigel
LASTNAME=Poulton
```

A drawback to using ConfigMaps with environment variables is that environment variables are static. This means updates made to the map are not reflected in running containers. For example, if you update the values of the `given` and `family` entries in the ConfigMap, environment variables in existing containers won't see the updates. This is a major reason environment variables aren't very good.

ConfigMaps and container startup commands

The concept of using ConfigMaps with container startup commands is simple. You specify a startup command for a container, and then customize it with variables.

The following Pod template is from the `startuppod.yml` file and describes a single container called `args1`. It's based on the `busybox` image and runs the `/bin/sh` startup command on line 5.

```
spec:
  containers:
    - name: args1
      image: busybox
      command: [ "/bin/sh", "-c", "echo First name $(FIRSTNAME) last name $(LASTNAME)" ]
      env:
        - name: FIRSTNAME
          valueFrom:
            configMapKeyRef:
              name: multimap
              key: given
        - name: LASTNAME
          valueFrom:
            configMapKeyRef:
              name: multimap
              key: family
```

If you look closely at the startup command, you'll see that it references two variables; `FIRSTNAME` and `LASTNAME`. Each of these is defined in the `env:` section directly below the startup command.

- `FIRSTNAME` is based on the `given` entry in the `multimap` ConfigMap
- `LASTNAME` is based on the `family` entry in the same ConfigMap

The relationship is shown in in Figure 11.3.

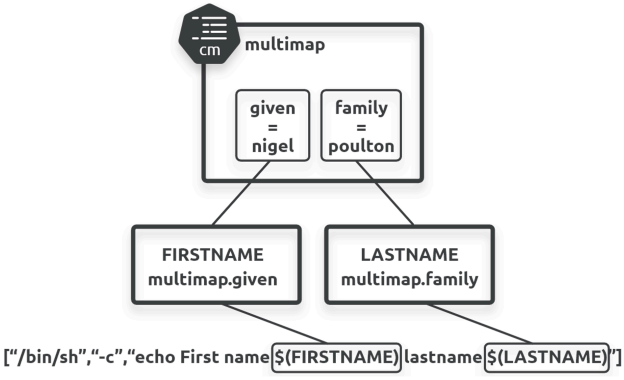

Figure 11.3

Running a Pod based on the previous YAML will print "First name Nigel last name Poulton" to the container's logs and then quit (succeed).

```
$ kubectl logs startup-pod -c args1
First name Nigel last name Poulton
```

Describing the Pod will yield the following lines describing the environment of the Pod.

```
Environment:
  FIRSTNAME:  <set to the key 'given' of config map 'multimap'>
  LASTNAME:  <set to the key 'family' of config map 'multimap'>
```

Using ConfigMaps with container startup commands is an extension of environment variables. As such, it suffers from the same limitations – updates to entries in the map will not be reflected in running containers.

If you ran the `startup-pod` it'll probably be in a `CrashLoopBackOff` loop. This is because it's startup command runs and completes, causing the Pod to succeed. Delete it with `kubectl delete pod startup-pod`.

ConfigMaps and volumes

Using ConfigMaps with volumes is the most flexible option. You can reference entire configuration files, as well as make updates to the ConfigMap and have them reflected in running containers. This means you can make changes to entries in a ConfigMap, after you've deployed a container, and those changes be seen in the container and available for running applications. The updates may take a minute or so to appear in the container.

The high-level process for exposing ConfigMap data via a volume looks like this.

1. Create the ConfigMap
2. Create a *ConfigMap volume* in the Pod template
3. Mount the *ConfigMap volume* into the container
4. Entries in the ConfigMap will appear in the container as individual files

This process is shown in Figure 11.4

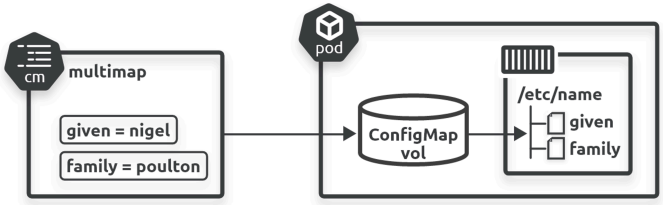

Figure 11.4

You still have the `multimap` ConfigMap with two values.

- given=Nigel
- family=Poulton

The following YAML creates a Pod called `cmvol` with the following configuration.

- `spec.volumes` creates a volume called **volmap** based on the **multimap** ConfigMap
- `spec.containers.volumeMounts` mounts the **volmap** volume to `/etc/name`

```
apiVersion: v1
kind: Pod
metadata:
  name: cmvol
spec:                           <<=== This block creates a
  volumes:                      <<=== special type of volume called
    - name: volmap              <<=== a ConfigMap volume based on
      configMap:                <<=== the ConfigMap called
        name: multimap          <<=== "multimap"
  containers:
    - name: ctr
      image: nginx
      volumeMounts:             <<=== These lines mount the
        - name: volmap          <<=== the "volmap" volume into the
          mountPath: /etc/name  <<=== container at "/etc/name"
```

The following commands deploy the container (from the `cmvol.yml` manifest) and then run a `kubectl exec` command to list the files in the '/etc/name/ directory.

```
$ kubectl apply -f cmpod.yml
pod/cmvol created

$ kubectl exec cmvol -- ls /etc/name
family
given
```

Edit the ConfigMap and change any of the values in the data block, you can even change them all. This may require a basic understanding of the `vi` editor. If you're not comfortable with `vi` you can edit the YAML file in a different editor and use `kubectl apply` to re-post it to the API server.

```
$ kubectl edit cm multimap

# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving
# this file will be reopened with the relevant failures.
#
apiVersion: v1
data:
  City: Macclesfield       << changed
  Country: UK              << changed
kind: ConfigMap
metadata:
<Snip>
```

Save your changes and re-run the previous `kubectl exec` command to list the contents of the container's filesystem. It may take a minute for the changes to be updated in the running container.

```
$ kubectl exec cmvol -- ls /etc/name
City
Country

$ kubectl exec cmvol -- cat /etc/name/Country
UK
```

Congratulations, the contents of the `multimap` ConfigMap have been exposed into the containers filesystem via ConfigMap volume and you've tested making an edit.

Hands-on with Secrets

Secrets are almost identical to ConfigMaps – they hold application configuration data that is injected into containers at run-time. However, Secrets are designed for sensitive data such as passwords, certificates, and OAuth tokens.

Are Secrets secure

The quick answer to this question is "no". But here's the slightly longer answer…

Despite being designed for sensitive data, Kubernetes does not encrypt Secrets. It merely obscures them as base-64 encoded values that can easily be decoded. Fortunately, it's possible to configure encryption-at-rest with `EncryptionConfiguration` objects, and most service meshes encrypt network traffic.

A typical workflow for a Secret is as follows.

1. The Secret is created and persisted to the cluster store as an un-encrypted object
2. A Pod that uses it gets scheduled to a cluster node
3. The Secret is transferred over the network, un-encrypted, to the node
4. The kubelet on the node starts the Pod and its containers
5. The Secret is mounted into the container via an in-memory `tmpfs` filesystem and decoded from base64 to plain text
6. The application consumes it
7. If/when the Pod is deleted, the Secret is deleted form the node

While it's possible to encrypt the Secret in the cluster store and leverage a service mesh to encrypt it in-flight on the network, it's always mounted as plain-text in the Pod/container. This is so the app can consume it without having to perform decryption or base64 decoding operations.

Also, use of in-memory `tmpfs` filesystems mean they're never persisted to disk on a node.

So, to cut a long story short, no Secrets aren't very secure. But you can take extra steps to make them secure.

They're also limited to 1MiB (1,048,576 bytes) in size.

An obvious use-case for Secrets is a generic TLS termination proxy for use across your `dev`, `test`, and `prod` environments. You create a standard image, and load the appropriate TLS keys at run-time for each environment. This is shown in the Figure 11.5.

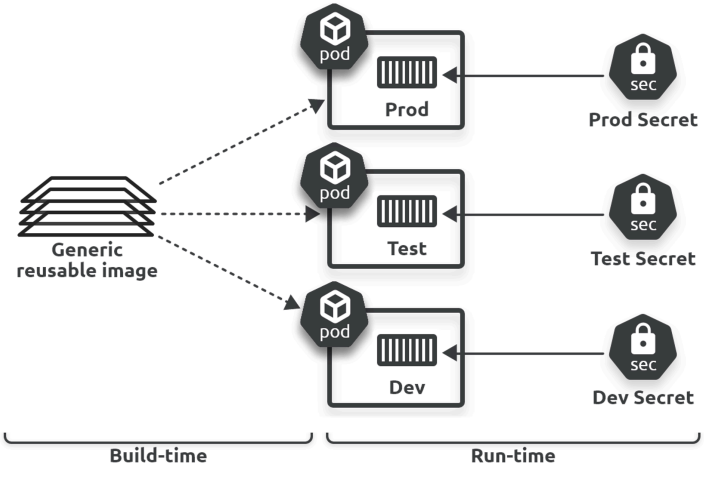

Figure 11.5

Inspecting system Secrets

By default, every Pod gets a Secret mounted into it as a volume, which it uses to authenticate itself if it talks to the API server.

If you've been following along, you'll still have the "cmvol" Pod running. Run the following command and identify the fields mapping the Secret to the Secret volume and container mount point.

```
1.  $ kubectl describe pod cmvol
2.  Name:         cmvol
3.  Namespace:    default
4.  <Snip>
5.  Containers:
6.    ctr:
7.      Container ID:   containerd://0de32d677251cbbda3ebe53e8...
8.      Image:          nginx
9.      Mounts:
10.       /etc/name from volmap (rw)
11.       /var/run/secrets/kubernetes.io/serviceaccount from default-token-s9nmx (ro)
12. <Snip>
13. Volumes:
14.   default-token-s9nmx:
15.     Type:       Secret (a volume populated by a Secret)
16.     SecretName: default-token-s9nmx
17.     Optional:   false
18. QoS Class:      BestEffort
19. <Snip>
```

Lines 13 - 17 define a *Secret volume* called "default-token-s9nmx" based on a Secret with the same name, and line 11 shows it mounted in the container at `/var/run/secrets/kubernetes.io/serviceaccount`.

You can see the Secret by listing all Secrets in the default Namespace. The name of yours will be different.

```
$ kubectl get secrets
NAME                  TYPE                                  DATA   AGE
default-token-s9nmx   kubernetes.io/service-account-token   3      21d
```

Describe the Secret to see more info.

```
$ kubectl describe secret default-token-s9nmx
Name:         default-token-s9nmx
Namespace:    default
Labels:       <none>
Annotations:  kubernetes.io/service-account.name: default
              kubernetes.io/service-account.uid: c5b5a4b3-3c5c...
Type:  kubernetes.io/service-account-token
Data
====
token:      eyJhbGciOiJSUzI1NiIsIm...
ca.crt:     570 bytes
namespace:  7 bytes
```

Creating Secrets

Before proceeding with this section, remember that Secrets are not encrypted in the cluster store, not encrypted in-flight on the network, and not encrypted when surfaced in a container. There are ways to encrypt them at-rest in the cluster store and to encrypt network traffic. However, they need to be surfaced in containers in plain text so applications can read them.

As with all API resources, Secrets can be created imperatively and declaratively.

Create a new Secret called **creds** with the following imperative command.

```
$ kubectl create secret generic creds --from-literal user=nigelpoulton \
  --from-literal pwd=Password123
```

Earlier you learned that Kubernetes obscures Secrets by encoding them as base64 values. Check this with the following command.

```
$ kubectl get secret creds -o yaml
apiVersion: v1
kind: Secret
data:
  pwd: UGFzc3dvcmQxMjM=
  user: bmlnZWxwb3VsdG9u
<Snip>
```

The username and password values are both base64-encoded. Run the following command to decode them. You'll need the `base64` utility on your system for the decoding to work. If you don't have it, you can use an online decoder.

```
$ echo UGFzc3dvcmQxMjM= | base64 -d
Password123
```

The decoding completes without requiring a key, proving that base64 encoding is not secure.

The following YAML object is from the `tkb-secret.yml` file in the `configmaps` folder. It describes a Secret called "tkb-secret" with two base64-encoded entries. If you want to add plain text entries, rename the `data` object to `stringData`. Although this allows you to enter values in plain text, they'll still be stored as base64, and subsequent `kubectl` commands will retrieve them as base64.

```
apiVersion: v1
kind: Secret
metadata:
  name: tkb-secret
  labels:
    chapter: configmaps
type: Opaque
data:                   <<=== Change to stringData for plain text values
  username: bmlnZWxwb3VsdG9u
  password: UGFzc3dvcmQxMjM=
```

Deploy it to your cluster. Be sure to run the command from the `configmaps` folder.

```
$ kubectl apply -f tkb-secret.yml
secret/tkb-secret created
```

Run `kubectl get` and `kubectl describe` commands to inspect it.

Using Secrets in Pods

The most flexible way to inject a Secret into a Pod (container) is via a special type of volume called a *Secret volume*.

The following YAML describes a single-container Pod with a Secret volume called "secret-vol" based on the `tkb-secret` created in the previous step. It also mounts it into the container at `/etc/tkb`.

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-pod
  labels:
    topic: secrets
spec:
  volumes:
  - name: secret-vol
    secret:
      secretName: tkb-secret
  containers:
  - name: secret-ctr
    image: nginx
    volumeMounts:
    - name: secret-vol
      mountPath: "/etc/tkb"
```

Secret vols are automatically mounted as read-only to prevent containers and applications accidentally mutating them.

Deploy it with the following command. Doing this will transfer the unencrypted Secret over the network to the kubelet on the node that will run the Pod where it will be mounted into the Pod via a tmpfs mount.

```
$ kubectl apply -f secretpod.yml
pod/secret-pod created
```

The following command shows the Secret is mounted as two files at `/etc/tkb` – one file for each entry in the Secret.

```
$ kubectl exec secret-pod -- ls /etc/tkb
password
username
```

Showing the contents of either file will show the entries have been exposed in the container in plain text for use by applications.

```
$ kubectl exec secret-pod -- cat /etc/tkb/password
Password123
```

There are more ways to use ConfigMaps and Secrets, but you know enough now to get started.

Clean-up

Use `kubectl get` to list the Pods, ConfigMaps and Secrets deployed in the chapter, and delete them with `kubectl delete`.

Chapter Summary

ConfigMaps and Secrets are the Kubernetes native way of decoupling applications and config data.

They're both first-class object in the Kubernetes API and can be created and manipulated with the usual `kubectl apply`, `kubectl get`, and `kubectl describe` commands. ConfigMaps are designed for application configuration parameters and even entire configuration files, whereas Secrets are designed for sensitive configuration data.

Both can be injected into containers at run-time via various constructs, with volumes being the preferred method, as they allow updates to eventually be reflected in running containers.

Secrets are not encrypted by default in the cluster store or when in transit on the network.